

Reinforcement Learning

Vanilla Policy Gradient & REINFORCE

Naeemullah Khan

naeemullah.khan@kaust.edu.sa

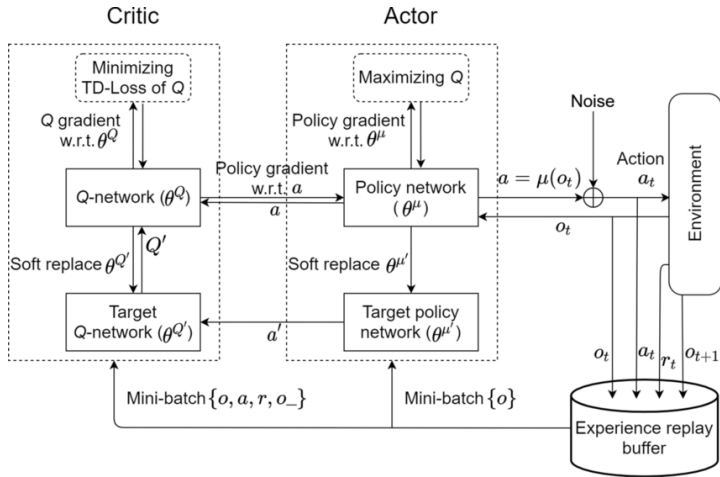


جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

June 14, 2026



Where we're heading: Actor-Critic architecture (Day 4)

1. Recap
2. Learning Outcomes
3. Motivation for Policy Gradient Methods
4. Policy Gradients
5. REINFORCE
6. Variance Reduction
7. On-Policy vs. Off-Policy Learning
8. REINFORCE in action: Recurrent Attention Model (RAM)
9. Policy Gradients in Action: AlphaGo Zero
10. Value-Based and Policy-Based RL
11. Policy Gradient vs. Q-Learning
12. Summary
13. References

- ▶ A Markov Decision Process (MDP) defines the RL problem using:
 - States $\mathcal{S}$
 - Actions $\mathcal{A}$
 - Rewards $\mathcal{R}$
 - Transition probabilities $\mathbb{P}$
 - Discount factor γ
- ▶ Markov property: The future depends only on the present state, not the past.
- ▶ Agent and environment interact in a loop.
- ▶ Policy π decides the agent's actions.

- ▶ Value function: Measures how good a state is.
- ▶ Q-value function: Measures how good a state-action pair is.
- ▶ Bellman equation: Recursively defines value and Q-value functions.
- ▶ Q-learning: Updates Q-values to reduce Bellman error.
- ▶ Deep Q-Learning: Uses neural networks to approximate Q-values.
- ▶ SARSA: On-policy version of Q-learning (learns from actions the policy actually takes).
- ▶ Deterministic vs. Stochastic policy: $\pi(s) = a$ vs. $\pi(a|s) = \Pr(a_t = a \mid s_t = s)$.

- Advantage function: $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ — how much better an action is than average in that state.

By the end of this session, you will be able to:

- ▶ Understand the **limitations of value-based methods** like Q-learning.
- ▶ Formally define and derive the **Policy Gradient** objective.
- ▶ Implement and interpret the **REINFORCE algorithm**.
- ▶ Explain the **on-policy vs. off-policy** distinction and its consequences for sample efficiency.
- ▶ Evaluate policy-based methods in different environments.

- ▶ Recall from Day 1: RL algorithms split into **value-based** and **policy-based** methods.
- ▶ Days 1–2 covered the value-based branch: Q-learning, DQN, SARSA.
- ▶ Today we move to the other branch: **directly optimizing the policy**.

- ▶ Recall from Day 1: RL algorithms split into **value-based** and **policy-based** methods.
- ▶ Days 1–2 covered the value-based branch: Q-learning, DQN, SARSA.
- ▶ Today we move to the other branch: **directly optimizing the policy**.
- ▶ Value-based methods learn Q-values and derive the policy indirectly.
 - Inefficient in continuous or large action spaces
 - Can't represent stochastic policies
 - May lead to high variance and instability

Policy Gradient Methods:

- Learn policy parameters directly to maximize expected return:

$$\mathcal{J}(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t \geq 0} \gamma^t r_t \right]$$

Reinforcement Learning: **Policy Gradients**

- ▶ **What is the problem with Q-learning?**
- ▶ The Q-function can be very complex.
- ▶ For example, a robot grasping an object may have a very high-dimensional state space. It can be difficult to learn the exact Q-value for every (state, action) pair.

- ▶ **What is the problem with Q-learning?**
- ▶ The Q-function can be very complex.
- ▶ For example, a robot grasping an object may have a very high-dimensional state space. It can be difficult to learn the exact Q-value for every (state, action) pair.
- ▶ However, the policy itself can be much simpler; for instance, just closing the robot's hand.
- ▶ Can we learn a policy directly, i.e., find the best policy from a set of possible policies?

- Formally, let us define a class of parameterized policies:

$$\Pi = \{\pi_{\theta} \mid \theta \in \mathbb{R}^m\}$$

- For each policy, we can define its expected return:

$$\mathcal{J}(\theta) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid \pi_{\theta} \right]$$

- Formally, let us define a class of parameterized policies:

$$\Pi = \{\pi_{\theta} \mid \theta \in \mathbb{R}^m\}$$

- For each policy, we can define its expected return:

$$\mathcal{J}(\theta) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid \pi_{\theta} \right]$$

- Our goal is to find the optimal policy: $\theta^* = \arg \max_{\theta} \mathcal{J}(\theta)$
- How can we achieve this?

- Formally, let us define a class of parameterized policies:

$$\Pi = \{\pi_{\theta} \mid \theta \in \mathbb{R}^m\}$$

- For each policy, we can define its expected return:

$$\mathcal{J}(\theta) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid \pi_{\theta} \right]$$

- Our goal is to find the optimal policy: $\theta^* = \arg \max_{\theta} \mathcal{J}(\theta)$
- How can we achieve this?
- **Solution:** Perform gradient ascent on the policy parameters!

- **Discrete actions** — Softmax policy:

$$\pi_\theta(a | s) = \frac{e^{\phi(s,a)^\top \theta}}{\sum_{a'} e^{\phi(s,a')^\top \theta}}$$

The network outputs one logit per action; softmax converts them to probabilities.

- **Discrete actions** — Softmax policy:

$$\pi_\theta(a | s) = \frac{e^{\phi(s,a)^\top \theta}}{\sum_{a'} e^{\phi(s,a')^\top \theta}}$$

The network outputs one logit per action; softmax converts them to probabilities.

- **Continuous actions** — Gaussian policy:

$$\pi_\theta(a | s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s)^2)$$

The network outputs a mean and standard deviation; actions are sampled from the resulting Gaussian.

- **Discrete actions** — Softmax policy:

$$\pi_\theta(a | s) = \frac{e^{\phi(s,a)^\top \theta}}{\sum_{a'} e^{\phi(s,a')^\top \theta}}$$

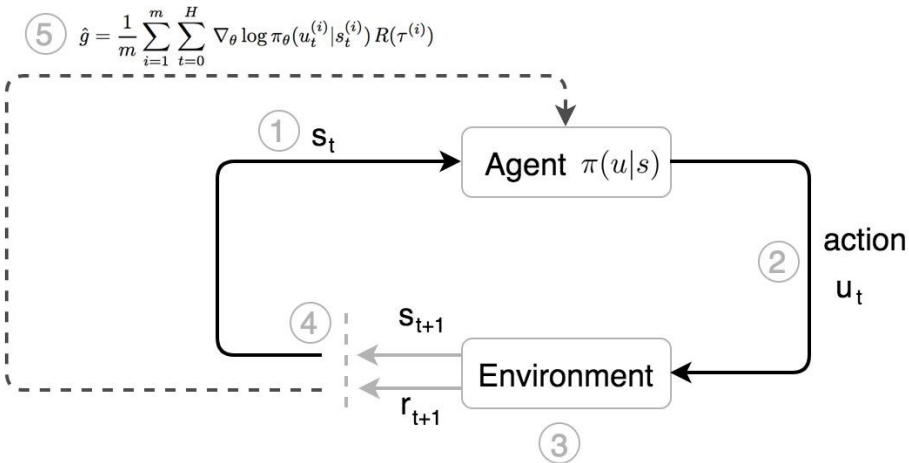
The network outputs one logit per action; softmax converts them to probabilities.

- **Continuous actions** — Gaussian policy:

$$\pi_\theta(a | s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s)^2)$$

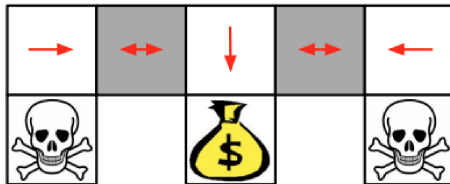
The network outputs a mean and standard deviation; actions are sampled from the resulting Gaussian.

- The Gaussian policy will be central to continuous-control algorithms (Days 5–6).
- Both are differentiable w.r.t. θ , which is what makes gradient ascent possible.



⁰Note: this diagram uses u for actions and $\pi(u|s)$; equivalent to a and $\pi(a|s)$ throughout this course.

Why Stochastic Policies? The Aliased Grid World



- ▶ Grey states are **aliased**: the agent observes the same features in multiple distinct positions
- ▶ A deterministic policy must make the same choice in every grey state — always go East or always go West — causing the agent to get stuck in a loop and never reach the goal
- ▶ A **stochastic** 50/50 policy breaks the symmetry: the agent eventually reaches the goal with probability 1
- ▶ This is the concrete motivation for the softmax and Gaussian parameterisations on the previous slide

Reinforcement Learning: **REINFORCE**

- ▶ REINFORCE is an elegant algorithm for maximizing the expected return.
- ▶ Intuition: trial and error.
- ▶ Sample a trajectory τ . If you get a high reward, try to make it more likely; if you get a low reward, try to make it less likely.
- ▶ A trajectory is a sequence of states, actions, and rewards: $\tau = (s_0, a_0, r_0, s_1, a_1, \dots)$.

- Expected reward, where $r(\tau) := \sum_{t \geq 0} \gamma^t r_t$ is the discounted trajectory return:

$$\begin{aligned}\mathcal{J}(\theta) &= \mathbb{E}_{\tau \sim p(\tau; \theta)} [r(\tau)] \\ &= \int_{\tau} r(\tau) p(\tau; \theta) d\tau\end{aligned}$$

- Expected reward, where $r(\tau) := \sum_{t \geq 0} \gamma^t r_t$ is the discounted trajectory return:

$$\begin{aligned}\mathcal{J}(\theta) &= \mathbb{E}_{\tau \sim p(\tau; \theta)} [r(\tau)] \\ &= \int_{\tau} r(\tau) p(\tau; \theta) d\tau\end{aligned}$$

- Now let's differentiate this:

$$\nabla_{\theta} \mathcal{J}(\theta) = \int_{\tau} r(\tau) \nabla_{\theta} p(\tau; \theta) d\tau$$

where $p(\tau; \theta) = \prod_{t \geq 0} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)$

- Expected reward, where $r(\tau) := \sum_{t \geq 0} \gamma^t r_t$ is the discounted trajectory return:

$$\begin{aligned}\mathcal{J}(\theta) &= \mathbb{E}_{\tau \sim p(\tau; \theta)} [r(\tau)] \\ &= \int_{\tau} r(\tau) p(\tau; \theta) d\tau\end{aligned}$$

- Now let's differentiate this:

$$\nabla_{\theta} \mathcal{J}(\theta) = \int_{\tau} r(\tau) \nabla_{\theta} p(\tau; \theta) d\tau$$

$$\text{where } p(\tau; \theta) = \prod_{t \geq 0} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)$$

- But this is **intractable**!

- ▶ However, we can use a useful trick:

$$\begin{aligned}\nabla_{\theta} p(\tau; \theta) &= p(\tau; \theta) \frac{\nabla_{\theta} p(\tau; \theta)}{p(\tau; \theta)} \\ &= p(\tau; \theta) \nabla_{\theta} \log p(\tau; \theta)\end{aligned}$$

- ▶ However, we can use a useful trick:

$$\begin{aligned}\nabla_{\theta} p(\tau; \theta) &= p(\tau; \theta) \frac{\nabla_{\theta} p(\tau; \theta)}{p(\tau; \theta)} \\ &= p(\tau; \theta) \nabla_{\theta} \log p(\tau; \theta)\end{aligned}$$

- ▶ Now, if we substitute this back:

$$\begin{aligned}\nabla_{\theta} \mathcal{J}(\theta) &= \int_{\tau} (r(\tau) \nabla_{\theta} \log p(\tau; \theta)) p(\tau; \theta) d\tau \\ &= \mathbb{E}_{\tau \sim p(\tau; \theta)} [r(\tau) \nabla_{\theta} \log p(\tau; \theta)]\end{aligned}$$

- ▶ We can estimate this with Monte Carlo sampling.

► Recall,

$$p(\tau; \theta) = \prod_{t \geq 0} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)$$

► Recall,

$$p(\tau; \theta) = \prod_{t \geq 0} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)$$

► Thus,

$$\log p(\tau; \theta) = \sum_{t \geq 0} \log p(s_{t+1} | s_t, a_t) + \log \pi_{\theta}(a_t | s_t)$$

- Recall,

$$p(\tau; \theta) = \prod_{t \geq 0} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)$$

- Thus,

$$\log p(\tau; \theta) = \sum_{t \geq 0} \log p(s_{t+1} | s_t, a_t) + \log \pi_{\theta}(a_t | s_t)$$

- When differentiating:

$$\nabla_{\theta} \log p(\tau; \theta) = \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- Recall,

$$p(\tau; \theta) = \prod_{t \geq 0} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)$$

- Thus,

$$\log p(\tau; \theta) = \sum_{t \geq 0} \log p(s_{t+1} | s_t, a_t) + \log \pi_{\theta}(a_t | s_t)$$

- When differentiating:

$$\nabla_{\theta} \log p(\tau; \theta) = \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- It doesn't depend on the transition probabilities!

- Therefore, when sampling a trajectory τ , we can estimate $\nabla_{\theta} \mathcal{J}(\theta)$ as:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} r(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- Therefore, when sampling a trajectory τ , we can estimate $\nabla_{\theta} \mathcal{J}(\theta)$ as:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} r(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

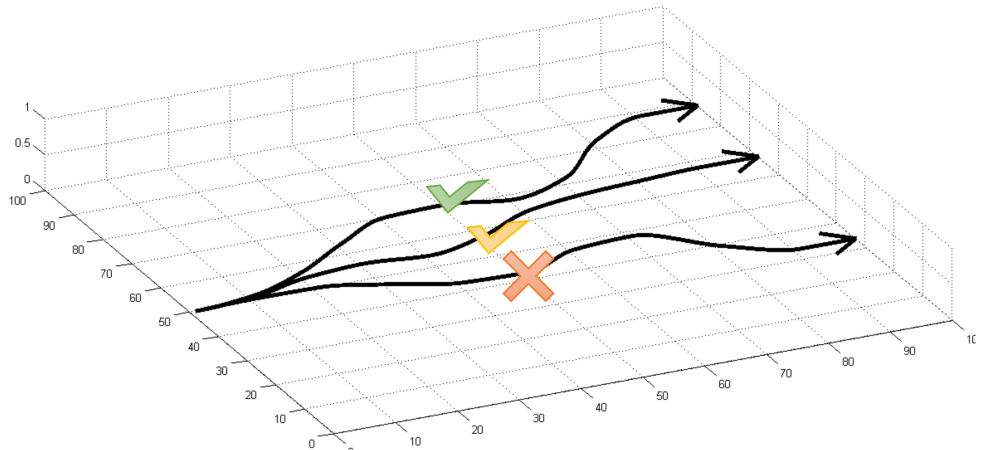
- Interpretation:

- If $r(\tau)$ is high, increase the probabilities of the actions taken.
- If $r(\tau)$ is low, decrease the probabilities of the actions taken.

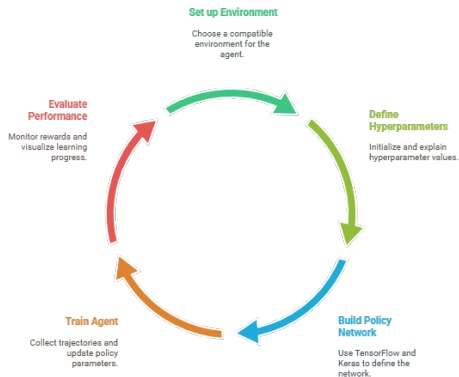
- Therefore, when sampling a trajectory τ , we can estimate $\nabla_{\theta} \mathcal{J}(\theta)$ as:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} r(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- Interpretation:
 - If $r(\tau)$ is high, increase the probabilities of the actions taken.
 - If $r(\tau)$ is low, decrease the probabilities of the actions taken.
- It might seem simplistic to say that if a trajectory is good, then all its actions were good. But in expectation, it averages out!



REINFORCE Algorithm Implementation Cycle



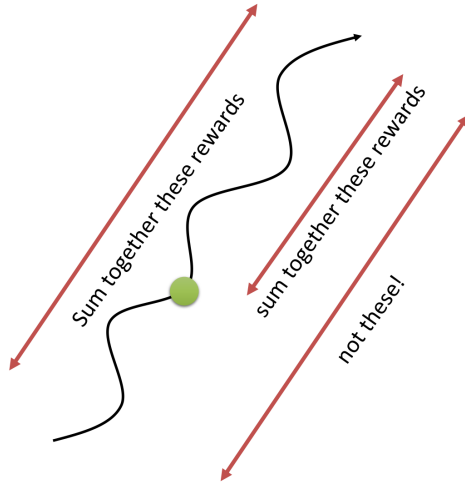
⁰Note: the diagram mentions TensorFlow/Keras; in this course we use PyTorch. The steps are framework-agnostic.

Reinforcement Learning: **Variance Reduction**

- ▶ However, there is a problem.
- ▶ This approach suffers from high variance because credit assignment is difficult.
- ▶ Can we help the estimator?

- ▶ However, there is a problem.
- ▶ This approach suffers from high variance because credit assignment is difficult.
- ▶ Can we help the estimator?
- ▶ **First idea — the Causality Trick:** Rewards before time t cannot be affected by action a_t , so they carry no useful gradient signal. Use only future rewards:

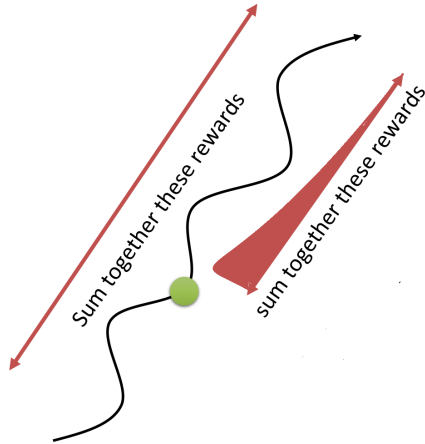
$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} \left(\sum_{t' \geq t} r_{t'} \right) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$



- ▶ The causality trick is **unbiased** — removing past rewards is mathematically correct.
- ▶ But Monte Carlo returns over full trajectories are still noisy — **high variance remains**.

- ▶ The causality trick is **unbiased** — removing past rewards is mathematically correct.
- ▶ But Monte Carlo returns over full trajectories are still noisy — **high variance remains**.
- ▶ **Second idea:** Weight future rewards by $\gamma^{t'-t}$, so distant rewards contribute less. This introduces a small **bias** but significantly reduces variance:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} \left(\sum_{t' \geq t} \gamma^{t'-t} r_{t'} \right) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$



- **Problem:** The raw value of a trajectory isn't necessarily meaningful. For example, if all rewards are positive, you keep increasing the probabilities of actions.

- ▶ **Problem:** The raw value of a trajectory isn't necessarily meaningful. For example, if all rewards are positive, you keep increasing the probabilities of actions.
- ▶ **What is important then?** Whether a reward is better or worse than what you expect to get.

- ▶ **Problem:** The raw value of a trajectory isn't necessarily meaningful. For example, if all rewards are positive, you keep increasing the probabilities of actions.
- ▶ **What is important then?** Whether a reward is better or worse than what you expect to get.
- ▶ **Idea:** Introduce a baseline function dependent on the state:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} \left(\sum_{t' \geq t} \gamma^{t'-t} r_{t'} - b(s_t) \right) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- ▶ **Problem:** The raw value of a trajectory isn't necessarily meaningful. For example, if all rewards are positive, you keep increasing the probabilities of actions.
- ▶ **What is important then?** Whether a reward is better or worse than what you expect to get.
- ▶ **Idea:** Introduce a baseline function dependent on the state:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} \left(\sum_{t' \geq t} \gamma^{t'-t} r_{t'} - b(s_t) \right) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- ▶ A simple baseline: the moving average of rewards experienced so far from all trajectories.

- ▶ Can we choose a better baseline?
- ▶ Essentially, we want to increase the probability of an action from a state if this action was better than the **expected value** from that state.

- ▶ Can we choose a better baseline?
- ▶ Essentially, we want to increase the probability of an action from a state if this action was better than the **expected value** from that state.
- ▶ What does this remind you of?

- ▶ Can we choose a better baseline?
- ▶ Essentially, we want to increase the probability of an action from a state if this action was better than the **expected value** from that state.
- ▶ What does this remind you of?
- ▶ **Recall from Day 1:** We defined the **Advantage function**:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

- ▶ $A^{\pi}(s_t, a_t) > 0$: action a_t was better than average — increase its probability.
- ▶ $A^{\pi}(s_t, a_t) < 0$: action a_t was worse than average — decrease its probability.

- ▶ Using the advantage as our baseline gives us the estimator:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} A^{\pi}(s_t, a_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- ▶ This is the best possible baseline: it directly measures how much better an action was than the policy's own average.
- ▶ It is unbiased, and it provides lower variance than a fixed moving-average baseline.

How to estimate A^{π} in practice? We only need to learn V_{ϕ} : the TD residual $r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$ approximates $A^{\pi}(s_t, a_t)$, so Q^{π} never needs to be estimated explicitly. This is covered on Day 4.

Reinforcement Learning: **On-Policy** vs. **Off-Policy**

- ▶ Recall from Day 2: SARSA is **on-policy** — it updates using the action the current policy actually takes.
- ▶ Q-learning is **off-policy** — it updates using $\max_{a'} Q(s', a')$, regardless of what the policy does.

- ▶ Recall from Day 2: SARSA is **on-policy** — it updates using the action the current policy actually takes.
- ▶ Q-learning is **off-policy** — it updates using $\max_{a'} Q(s', a')$, regardless of what the policy does.
- ▶ **REINFORCE is also on-policy**: it samples full trajectories under the current policy π_θ and uses them to estimate the gradient.
- ▶ Once the policy parameters θ are updated, the old trajectories are discarded — they came from a different policy and cannot be reused.

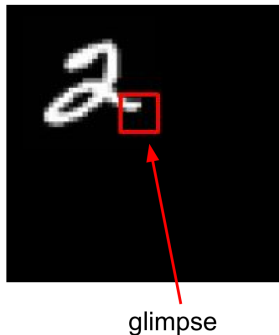
- ▶ Recall from Day 2: SARSA is **on-policy** — it updates using the action the current policy actually takes.
- ▶ Q-learning is **off-policy** — it updates using $\max_{a'} Q(s', a')$, regardless of what the policy does.
- ▶ **REINFORCE is also on-policy**: it samples full trajectories under the current policy π_θ and uses them to estimate the gradient.
- ▶ Once the policy parameters θ are updated, the old trajectories are discarded — they came from a different policy and cannot be reused.
- ▶ **Consequence**: REINFORCE cannot use an experience replay buffer.
 - Replay stores transitions from past policies — that is off-policy data.
 - Mixing off-policy data into an on-policy gradient estimate introduces bias.

- ▶ Recall from Day 2: SARSA is **on-policy** — it updates using the action the current policy actually takes.
- ▶ Q-learning is **off-policy** — it updates using $\max_{a'} Q(s', a')$, regardless of what the policy does.
- ▶ **REINFORCE is also on-policy:** it samples full trajectories under the current policy π_θ and uses them to estimate the gradient.
- ▶ Once the policy parameters θ are updated, the old trajectories are discarded — they came from a different policy and cannot be reused.
- ▶ **Consequence:** REINFORCE cannot use an experience replay buffer.
 - Replay stores transitions from past policies — that is off-policy data.
 - Mixing off-policy data into an on-policy gradient estimate introduces bias.
- ▶ This sample inefficiency is a core limitation of vanilla policy gradient methods and a key motivation for the algorithms we cover in Days 4 and 6.

REINFORCE in Action: **Recurrent Attention Model (RAM)**

- ▶ **Objective:** Image classification
- ▶ The model takes a sequence of “glimpses,” selectively focusing on regions of the image to predict the class.
 - Inspired by human perception and eye movements
 - Saves computational resources $\Rightarrow$ improves scalability
 - Can ignore clutter or irrelevant parts of the image
- ▶ **State:** Glimpses observed so far
- ▶ **Action:** (x, y) coordinates (center of the next glimpse) indicating where to look next in the image
- ▶ **Reward:** 1 at the final timestep if the image is correctly classified, 0 otherwise

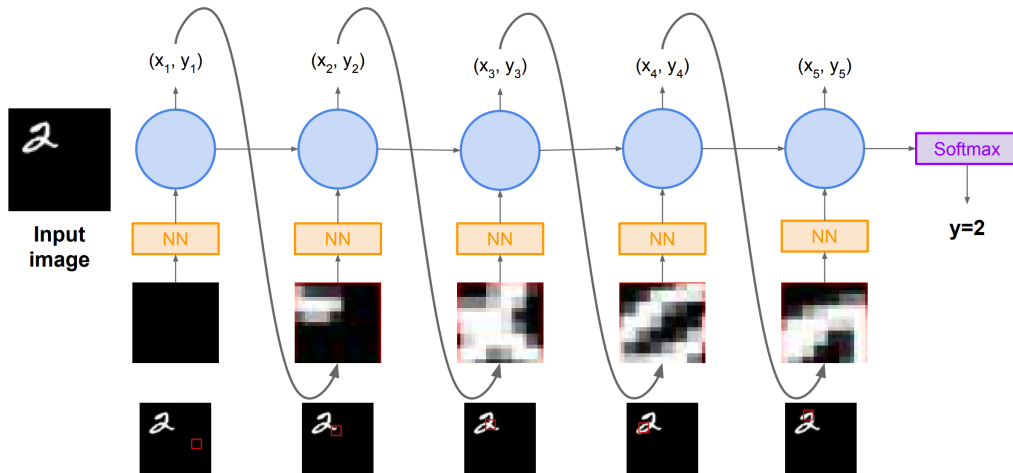
⁰[Mnih et al., 2014]



- ▶ Glimpsing is a non-differentiable operation.
- ▶ The policy for selecting glimpse locations is learned using REINFORCE.
- ▶ Given the sequence of glimpses observed so far, an RNN models the state and outputs the next action.

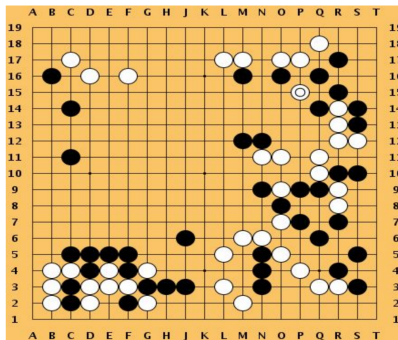
⁰[Mnih et al., 2014]

REINFORCE in Action: Recurrent Attention Model (RAM)



⁰[Mnih et al., 2014]

Policy Gradients in Action: **AlphaGo Zero**



- ▶ AlphaGo Zero (2017) reaches superhuman Go play with **no human data** — pure self-play using policy gradients
- ▶ A cleaner demonstration of REINFORCE-style learning than the original AlphaGo, which relied on supervised pretraining from human games

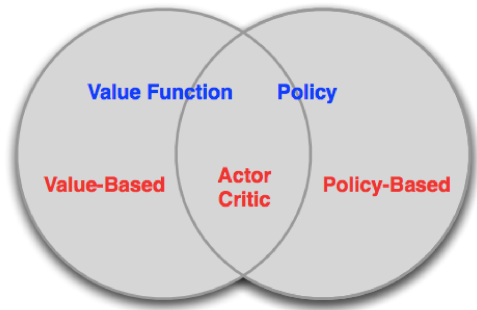
⁰Silver et al., *Mastering the game of Go without human knowledge*, Nature 550, 354–359 (2017)

- ▶ **No human games:** policy network initialised randomly; improves entirely through self-play
- ▶ **Policy gradient training:** actions sampled from the policy; wins (+1) and losses (−1) provide the reward signal
- ▶ **Value network:** learns to estimate board outcome from any position, acting as the critic
- ▶ **Monte Carlo Tree Search:** combines policy and value networks for lookahead at inference time
- ▶ Surpassed the original AlphaGo within 40 days of training

⁰Silver et al., *Mastering the game of Go without human knowledge*, Nature 550, 354–359 (2017)

Reinforcement Learning: **Value-Based** and **Policy-Based** Methods

- ▶ **Value-Based Methods**
 - Learn a value function
 - Derive policy implicitly (e.g., ϵ -greedy)
- ▶ **Policy-Based Methods**
 - Do not learn a value function
 - Learn the policy directly
- ▶ **Actor-Critic Methods** (*Day 4*)
 - Learn both a value function and a policy
 - Combines the strengths of both approaches above



- ▶ Policy gradient and Q-learning use two fundamentally different representations: policies and value functions.
- ▶ Advantage of both methods: no need to model the environment.

- ▶ Policy gradient and Q-learning use two fundamentally different representations: policies and value functions.
- ▶ Advantage of both methods: no need to model the environment.
- ▶ **Policy Gradient: Pros and Cons**
 - **Pros:** Unbiased estimate of the gradient of expected return.
 - Can handle large action spaces (since only one action needs to be sampled).
 - **Cons:** High variance updates (leads to poor sample efficiency).
 - Does not perform credit assignment effectively.

- ▶ Policy gradient and Q-learning use two fundamentally different representations: policies and value functions.
- ▶ Advantage of both methods: no need to model the environment.
- ▶ **Policy Gradient: Pros and Cons**
 - **Pros:** Unbiased estimate of the gradient of expected return.
 - Can handle large action spaces (since only one action needs to be sampled).
 - **Cons:** High variance updates (leads to poor sample efficiency).
 - Does not perform credit assignment effectively.
- ▶ **Q-Learning: Pros and Cons**
 - **Pros:** Lower variance updates, more sample efficient.
 - Performs credit assignment.
 - **Cons:** Biased updates due to function approximation.
 - Difficult to handle large action spaces (since the maximum over actions must be computed).

Reinforcement Learning: **Summary**

Method	Policy Type	Gradient Source	Stability	Efficiency
Q-learning	Deterministic	Value gradients	Medium	High
REINFORCE	Stochastic	Monte Carlo returns	Low	Low
Actor-Critic	Stochastic	TD residual (Day 4)	Medium	Medium

- ▶ **High variance** in gradient estimates — Monte Carlo returns are noisy.
- ▶ **Sample inefficiency** — on-policy: each trajectory is used once and discarded.
- ▶ **Slow convergence** — many samples needed before the signal-to-noise ratio is acceptable.
- ▶ **No credit assignment** across timesteps beyond the causality trick and discounting.
- ▶ Variance reduction techniques (baselines, advantage) help but do not fully solve these issues.

- ▶ **Day 4 — Actor-Critic & PPO:** Combine a learned critic V_ϕ with the policy gradient to reduce variance. Build up to A2C, TRPO, and PPO.
- ▶ **Day 5 — DDPG:** Extend to deterministic policies for continuous action spaces.
- ▶ **Day 6 — SAC:** Add an **entropy bonus** $\mathcal{H}(\pi_\theta(\cdot|s))$ to the reward signal — encouraging exploration while maximizing return.
- ▶ **Days 7–10:** Exploration, model-based planning, RLHF, and open problems.

- ▶ It can be hard to learn the exact Q-value for every (state, action) pair in high-dimensional state and action spaces.
- ▶ However, we can just learn a policy that maximizes the reward.
- ▶ We can use gradient ascent on policy parameters.
- ▶ However, this can suffer from high variance. Various strategies exist to tackle this: causality, discounting, baselines, and the advantage function.
- ▶ *Next: Day 4 introduces Actor-Critic methods, which learn a critic V_ϕ alongside the policy to make advantage estimation practical.*

Vanilla Policy Gradient: **References**

- [1] Sutton, R. S., & Barto, A. G. (2018).
Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

- [2] Williams, R. J. (1992).
Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning.
Machine Learning, 8, 229–256.

- [3] Sutton, R. S., McAllester, D. A., Singh, S. P., & Mansour, Y. (2000).
Policy Gradient Methods for Reinforcement Learning with Function Approximation.
In *Advances in Neural Information Processing Systems (NeurIPS)* 12, 1057–1063.

- [4] Silver, D., Schrittwieser, J., Simonyan, K., et al. (2017).
Mastering the Game of Go without Human Knowledge.
Nature, 550, 354–359.

- [5] Mnih, V., Heess, N., Graves, A., & Kavukcuoglu, K. (2014).
Recurrent Models of Visual Attention.
In *Advances in Neural Information Processing Systems (NeurIPS)* 27.

- [6] OpenAI Spinning Up.
<https://spinningup.openai.com>

- [7] Sergey Levine, Berkeley CS285: Deep Reinforcement Learning.
<https://rail.eecs.berkeley.edu/deeprlcourse/>

- [8] Chelsea Finn & Karol Hausman, Stanford CS224R: [Deep Reinforcement Learning](#).